

Media to Text AI Project: Learning and Build Plan

We are not training any models. Every model we use is already trained, so the work is inference and integration.

What needs to be learned

Taking in specific video, audio, and image files, and finding the proper Python libraries to handle speech to text or image to text. The correct package names are torchaudio and torchvision, not PyTorch audio and PyTorch vision. Neither one does transcription or captioning on its own: torchaudio loads and transforms audio, torchvision handles image loading and classification. For actual speech to text the tool is Whisper, and for image to text a vision language model through Hugging Face transformers.

If using the Gemini API or another AI provider, how it handles requests and what its limits are. Implement a rate limit. This is two pieces of work: a throttle that spaces our outgoing requests, and retry with backoff for when the API returns a 429 anyway.

Learning HTML, CSS, and JavaScript for the GUI, plus proper design and structure. HTML is structure, CSS is styling, and JavaScript is where functions live. The server versus file question is really web app with a Python backend versus local desktop app. The deciding factor is that an API key can never sit in browser side JavaScript, so a web GUI requires a backend to hold the key. Implement a file size limit, enforced on the backend rather than in the browser.

Open question: Gemini accepts media directly, so if the API handles the media step we do not need torchaudio, torchvision, or Whisper at all. Decide this before anyone goes deep on either track.

Who learns each thing, from which sources, and by what date

Christian: Gather information on multiple AI APIs, note which works best for the project, and find cost friendly options. Cover accessing API keys, their respective rate limits and quotas, integrating the model into our program, and good practice for handling keys. By October 1.

Cooper: Has HTML and JS experience, stronger in Python. Unfamiliar with APIs. Plans to read documentation and watch tutorials on implementation and good practices. Model training is off the list since the project does not involve it. By September 30.

Will: Whisper and torchaudio implementation in our code, and calling APIs from other products where needed. By October 1.

Jared: Familiar with self-hosting, the gap is in how files are ingested to the server and processed to the API. Will learn how to get files securely uploaded to the server and sent to the correct places. Will learn this by October 1. Will learn Flask as a starting frame work

Aaron: Familiar with self hosting and multi step transformer backends in Python. Gap is that the self hosting experience is solo, so the target is how it translates to team work and what good practices to pick up. By September 20.

Who builds each part of the skeleton, who approves it, and by what date

Approvers and build dates still need filling in.

Christian: Build the connection between the AI model and the program.

Cooper: Build the GUI and any Python scripting for interaction between the GUI and the application.

Will: Python code within the application to connect everything together. Pytorch.

Jared: Build and maintain the backend web services for the application

Aaron: Build the media ingestion pipeline. Take files after Jared's upload step, validate them, normalize them (extract audio from video, convert formats, chunk long audio, resize images), and hand clean inputs to the model layer.

Not yet owned: integration and testing.

Backup

API integration and implementation: Christian and Will

Python scripting and media library implementation: Cooper and Will

Backend: Jared and Aaron

Backend still needs defining. It could mean the server, the API layer, or the local model layer.

Teaching the team

Weekly team meetings, in person or virtual, with discussion of workflow and progress checks. Worth adding: a short walkthrough from each person once their learning date passes.